



University Of Westminster

Client Server Architecture

5COSC022W

Test Report for the Bookstore APIs

Full Name: Bamunusingha Disandu Sanhida Jayawardhana

UOW Number: W2083055

IIT Number: 20230469

Tutorial Group: Group 20

Submission Date: 28 April 2025

Assignment: Bookstore API with JAX-RS

01. Introduction

The BookStore API Test Report summarizes the extensive test work done on the Bookstore API, a web service designed to manage bookstore activity like books, authors, customers, shopping carts, and orders. The primary purpose of this report is to present an overview of a comprehensive series of test cases that verify the API's behavior, stability, and exception handling in various scenarios. By testing each endpoint methodically, the report ensures that the API is designed to specification and operates reliably in a variety of conditions, including valid input, invalid requests, and edge cases.

The API was developed using JAX-RS (Java API for RESTful Web Services), leveraging its scalability and maintainability capabilities in developing RESTful endpoints. Data exchange between the API is done using JSON (JavaScript Object Notation) to facilitate lightweight and structured communication between the server and the clients. To ensure the functionality of the API, all the testing was performed using Postman, a multifaceted API testing tool which can send HTTP requests, validate the response, and automate test cases. Postman's simplicity allowed for developing a comprehensive test suite containing all the API endpoints with precision.

This report is intended to present test cases in an unambiguous, tabular form so that it is easy to understand the goal, execution, and outcome of each test. Each table includes significant details such as the endpoint, HTTP method, request body (in JSON), expected HTTP status code, expected response body (in JSON), and actual test result. The test cases are organized by resource (e.g., BookResource, AuthorResource, CustomerResource, CartResource, OrderResource) and cover a range of scenarios, including successful operations, error conditions, and boundary cases. This structured approach ensures that the API's functionality is thoroughly validated, providing confidence in its deployment readiness.

The remainder of the report is dedicated to the detailed test case tables, which serve as both documentation and evidence of the API's performance. By combining rigorous testing with clear presentation, this report aims to demonstrate the BookStore API's adherence to RESTful principles and its ability to handle real-world bookstore management tasks effectively.

02. API Endpoint Test Case Tables

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /books	Create book with valid data	POST	{"title": "The Hobbit", "author": "J.R.R. Tolkien", "isbn": "978-0-261-10221-7", "year": 1937, "price": 15, "stockQuantity": 50}	201 Created	{"bookId": "<generated_id>", "title": "The Hobbit", "author": "J.R.R. Tolkien", "isbn": "978-0-261-10221-7", "year": 1937, "price": 15, "stockQuantity": 50}	Pass
POST /books	Create book with missing ISBN	POST	{"title": "The Hobbit", "author": "J.R.R. Tolkien", "year": 1937, "price": 15, "stockQuantity": 50}	400 Bad Request	{"error": "InvalidInputException", "message": "ISBN is required"}	Pass
POST /books	Create book with future year	POST	{"title": "The Hobbit", "author": "J.R.R. Tolkien", "isbn": "978-0-261-10221-7", "year": 2057, "price": 15, "stockQuantity": 50}	400 Bad Request	{"error": "InvalidInputException", "message": "Publication year cannot be in the future"}	Pass
GET /books	Get all books	GET	None	200 OK	[{"bookId": "255aa6bd-ce3b-4a6d-921d-7e5239e044c6", "title": "The Hobbit", ...}, ...]	Pass

GET /books/{id}	Get book by valid ID	GET	None	200 OK	{"bookId": "255aa6bd-ce3b-4a6d-921d-7e5239e044c6", "title": "The Hobbit", ...}	Pass
GET /books/{id}	Get book by invalid ID	GET	None	404 Not Found	{"error": "BookNotFoundException", "message": "Book with ID 255aa6bd-ce3b-4a6d-921d not found"}	Pass
PUT /books/{id}	Update book with valid data	PUT	{"bookId": "255aa6bd-ce3b-4a6d-921d-7e5239e044c6", "title": "The Hobbit", "author": "J.R.R. Tolkien", "isbn": "978-0-261-10221-7", "year": 1937, "price": 20, "stockQuantity": 40}	200 OK	{"bookId": "255aa6bd-ce3b-4a6d-921d-7e5239e044c6", "title": "The Hobbit", ...}	Pass
PUT /books/{id}	Update book with invalid ID	PUT	{"bookId": "255aa6bd-ce3b-4a6d-9", "title": "The Hobbit", "author": "J.R.R. Tolkien", "isbn": "978-0-261-10221-7", "year": 1937, "price": 20, "stockQuantity": 40}	404 Not Found	{"error": "BookNotFoundException", "message": "Book with ID 255aa6bd-ce3b-4a6d-9 not found"}	Pass
DELETE /books/{id}	Delete book by valid ID	DELETE	None	204 No Content	None	Pass
DELETE /books/{id}	Delete book by invalid ID	DELETE	None	404 Not Found	{"error": "BookNotFoundException", "message": "Book with ID 255aa6bd-ce3b-4a6d-921d not found"}	Pass

POST /authors	Create author with valid data	POST	{"name": "J.K. Rowling", "biography": "British author, best known for the Harry Potter series"}	201 Created	{"authorId": "<generated_id>", "name": "J.K. Rowling", "biography": "British author, best known for the Harry Potter series"}	Pass
POST /authors	Create author with missing name	POST	{"biography": "British author, best known for the Harry Potter series"}	400 Bad Request	{"error": "InvalidInputException", "message": "Name is required"}	Pass
GET /authors	Get all authors	GET	None	200 OK	[{"authorId": "25d81618-4887-4b62-b953-437e47a04a2f", "name": "J.K. Rowling", ...}, ...]	Pass
GET /authors/{id}	Get author by valid ID	GET	None	200 OK	{"authorId": "25d81618-4887-4b62-b953-437e47a04a2f", "name": "J.K. Rowling", ...}	Pass
GET /authors/{id}	Get author by invalid ID	GET	None	404 Not Found	{"error": "AuthorNotFoundException", "message": "Author with ID 25d81618-4887-4b62-b953-437e47a not found"}	Pass
GET /authors/{id}/books	Get books by valid author ID	GET	None	200 OK	[{"bookId": "255aa6bd-ce3b-4a6d-921d-7e5239e044c6", "title": "The Hobbit", ...}, ...]	Pass
GET /authors/{id}/books	Get books by invalid author ID	GET	None	404 Not Found	{"error": "AuthorNotFoundException", "message": "Author with ID 25d81618-4887-4b62-b953-437e47a not found"}	Pass

PUT /authors/{id}	Update author with valid data	PUT	{"name": "J.K. Rowling", "biography": "Updated Biography: British author, best known for the Harry Potter Writer"}	200 OK	{"authorId": "a9e5a118-e4a4-438e-8195-80e1e85016c7", "name": "J.K. Rowling", ...}	Pass
PUT /authors/{id}	Update author with invalid ID	PUT	{"name": "Disandu Sanhida", "biography": "American novelist and short story writer, best known for the epic fantasy series A Song of Ice and Fire."}	404 Not Found	{"error": "AuthorNotFoundException", "message": "Author with ID a9e5a118-e4a4-438e-8195-80e1e85016c7 not found"}	Pass
DELETE /authors/{id}	Delete author by valid ID	DELETE	None	204 No Content	None	Pass
DELETE /authors/{id}	Delete author by invalid ID	DELETE	None	404 Not Found	{"error": "AuthorNotFoundException", "message": "Author with ID a9e5a118-e4a4-438e-8195-80e1e85016c7 not found"}	Pass
POST /customers	Register customer with valid data	POST	{"name": "Disandu Sanhida", "email": "disandu@gmail.com", "password": "pass123we"}	201 Created	{"customerId": "<generated_id>", "name": "Disandu Sanhida", "email": "disandu@gmail.com", ...}	Pass
POST /customers	Register customer with missing email	POST	{"name": "John Doe", "password": "pass123"}	400 Bad Request	{"error": "InvalidInputException", "message": "Email is required"}	Pass

GET /customers	Get all customers	GET	None	200 OK	{ "customerId": "e3afd0dd-9af4-4a15-9658-ce5022bb2afa", "name": "Disandu Sanhida", ...}, ...}	Pass
GET /customers/{id}	Get customer by valid ID	GET	None	200 OK	{ "customerId": "e3afd0dd-9af4-4a15-9658-ce5022bb2afa", "name": "Disandu Sanhida", ...}	Pass
GET /customers/{id}	Get customer by invalid ID	GET	None	404 Not Found	{ "error": "CustomerNotFoundException", "message": "Customer with ID e3afd0dd-9af4 not found"} }	Pass
PUT /customers/{id}	Update customer with valid data	PUT	{ "customerId": "e3afd0dd-9af4-4a15-9658-ce5022bb2afa", "name": "Disandu Sanhida", "email": "disandu@example.com", "password": "pass123"} }	200 OK	{ "customerId": "e3afd0dd-9af4-4a15-9658-ce5022bb2afa", "name": "Disandu Sanhida", ...}	Pass
PUT /customers/{id}	Update customer with invalid ID	PUT	{ "customerId": "e3afd0dd-9af4-4a15-9658", "name": "Disandu Sanhida", "email": "disandu@example.com", "password": "pass123"} }	404 Not Found	{ "error": "CustomerNotFoundException", "message": "Customer with ID e3afd0dd-9af4-4a15-9658 not found"} }	Pass
DELETE /customers/{id}	Delete customer by valid ID	DELETE	None	204 No Content	None	Pass

DELETE /customers/{id}	Delete customer by invalid ID	DELETE	None	404 Not Found	{"error": "CustomerNotFoundException", "message": "Customer with ID e3afd0dd-9af4-4a15-9658 not found"}	Pass
POST /customers/{customerId}/cart	Add item to cart with valid data	POST	{"items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 10}}	201 Created	{"cartId": "<generated_id>", "items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 10}}	Pass
POST /customers/{customerId}/cart	Add item with invalid book ID	POST	{"items": {"book1": 2}}	400 Bad Request	{"error": "InvalidInputException", "message": "Invalid book ID"}	Pass
POST /customers/{customerId}/cart	Add item with zero quantity	POST	{"items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 0}}	400 Bad Request	{"error": "InvalidInputException", "message": "Quantity must be greater than zero"}	Pass
GET /customers/{customerId}/cart	Get cart by valid customer ID	GET	None	200 OK	{"cartId": "<generated_id>", "items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 10, ...}}	Pass
GET /customers/{customerId}/cart	Get cart by invalid customer ID	GET	None	404 Not Found	{"error": "CartNotFoundException", "message": "Cart for customer d454624c-d1ea-4e58 not found"}	Pass
PUT /customers/{customerId}/cart/items/{bookId}	Update cart item quantity with valid data	PUT	{"items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 15}}	200 OK	{"cartId": "<generated_id>", "items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 15}}	Pass

PUT /customers/{customerId}/cart/items/{bookId}	Update cart with invalid book ID	PUT	{"items": {"eeadd9a0-a273-4728-99ed-f06f5de74ce3": 15}}	400 Bad Request	{"error": "InvalidInputException", "message": "Invalid book ID"}	Pass
PUT /customers/{customerId}/cart/items/{bookId}	Update cart with zero quantity	PUT	{"items": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 0}}	400 Bad Request	{"error": "InvalidInputException", "message": "Quantity must be greater than zero"}	Pass
DELETE /customers/{customerId}/cart/items/{bookId}	Remove item from cart with valid book ID	DELETE	None	204 No Content	None	Pass
DELETE /customers/{customerId}/cart/items/{bookId}	Remove item with invalid book ID	DELETE	None	400 Bad Request	{"error": "InvalidInputException", "message": "Invalid book ID"}	Pass
POST /customers/{customerId}/orders	Place order with valid data	POST	{"order": {"2888890f-6bbe-4635-8284-2ad5e2372f1c": 2}}	201 Created	{"orderId": "<generated_id>", "items": {"2888890f-6bbe-4635-8284-2ad5e2372f1c": 2}}	Pass
POST /customers/{customerId}/orders	Place order with invalid customer ID	POST	{"order": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 2}}	404 Not Found	{"error": "CustomerNotFoundException", "message": "Customer with ID c53cc8dc-ecc8-47ea14 not found"}	Pass
POST /customers/{customerId}/orders	Place order with zero quantity	POST	{"order": {"93292105-a4e9-4152-b8f8-ef1c9f239096": 0}}	400 Bad Request	{"error": "InvalidInputException", "message": "Quantity must be greater than zero"}	Pass

POST /customers/{customerId}/orders	Place order with out-of-stock item	POST	{"order": {"145f2f57-e38d-44cc-bbc9-0edd1f8087e4": 1000}}	400 Bad Request	{"error": "OutOfStockException", "message": "Book with ID 145f2f57-e38d-44cc-bbc9-0edd1f8087e4 is out of stock"}	Pass
GET /customers/{customerId}/orders	Get all orders by valid customer ID	GET	None	200 OK	[{"orderId": "<generated_id>", "items": {"2888890f-6bbe-4635-8284-2ad5e2372f1c": 2, ..., ...}]	Pass
GET /customers/{customerId}/orders	Get orders by invalid customer ID	GET	None	404 Not Found	{"error": "CustomerNotFoundException", "message": "Customer with ID d0158f20-f94a593a86ce94e not found"}	Pass
GET /customers/{customerId}/orders/{orderId}	Get order by valid customer and order ID	GET	None	200 OK	{"orderId": "2888890f-6bbe-4635-8284-2ad5e2372f1c", "items": {"2888890f-6bbe-4635-8284-2ad5e2372f1c": 2}}	Pass
GET /customers/{customerId}/orders/{orderId}	Get order by invalid order ID	GET	None	404 Not Found	{"error": "OrderNotFoundException", "message": "Order with ID 2888890f-6bbe not found"}	Pass